FAKULTI TEKNOLOGI DAN KEJURUTERAAN ELEKTRONIK DAN KOMPUTER

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

BERR 2243: DATABASE AND CLOUD SYSTEM

BERR 2143	SEMESTER 1	SESI 2025/2026

DESIGN AND SIMULATION OF AN AOI22 CMOS LOGIC GATE USING LTSPICE

NO.	STUDENTS' NAME	Matrix No.
1.	CHAN HAN LUO	B122410386
2.	CHAN SHI YAO	B122410388
3.	HOO YONG SHENG	B122410369
4.	NELSON WONG CHIEW XING	B122510869

PROGRAMME	2BERR
SECTION / GROUP	S1/GA
DATE	16 JUNE 2026
NAME OF LECTURER	DR. SOO YEW GUAN

EXAMINER'S COMMENT(S)	TOTAL MARKS

Abstract

This project presents the design and deployment of a cloud-based Internet of Things (IoT) home security monitoring system using an ESP32 microcontroller, cloud middleware services, MongoDB Atlas, and MongoDB Charts. The objective was to develop a complete cyber-physical data pipeline capable of collecting environmental data from edge devices, securely transmitting telemetry through MQTT, validating device authorization, storing data in MongoDB Atlas, and visualizing real-time information through a dashboard.

1. Introduction

This project implemented a complete end-to-end Internet of Things (IoT) platform for home security monitoring using an ESP32 microcontroller, cloud middleware services, MongoDB Atlas, and MongoDB Charts. The objective was to design a cyber-physical system capable of collecting sensor data at the edge layer, securely transmitting telemetry through MQTT, validating device authorization, storing data in a cloud database, and visualizing information through a real-time dashboard.

The developed system uses an ESP32 FM DevKit integrated with an HC-SR04 ultrasonic sensor and an MH-B infrared (IR) sensor. The ultrasonic sensor continuously monitors the surrounding environment while the IR sensor verifies object presence and movement. When predefined conditions are met, telemetry data are published to a cloud-hosted MQTT broker. The data are processed by Node-RED, validated against registered device information, and stored in MongoDB Atlas before being displayed through MongoDB Charts.

The project demonstrates practical implementation of cloud computing, NoSQL databases, MQTT communication, containerized deployment, and IoT security principles.

2. System Deployment Process

2.1 Edge Layer Deployment

The edge device consists of an ESP32 FM DevKit connected to an HC-SR04 ultrasonic sensor, an MH-B IR sensor, and an LED indicator.GPIO assignments are shown as Table 1:

Component	GPIO
Ultrasonic Trigger	GPIO 18
Ultrasonic Echo	GPIO 19
IR Sensor	GPIO 5
LED Output	GPIO 23

Table 1

The ESP32 connects to a Wi-Fi network and establishes a secure MQTT connection with the EMQX broker using TLS encryption on port 8883. Sensor data are formatted as JSON payloads and published to the MQTT topic home/telemetry.The ESP32 also subscribes to the home/command topic, enabling bidirectional communication and remote control of the LED indicator.

2.2 Cloud Middleware Deployment

A Linux Virtual Machine hosted on Microsoft Azure was provisioned to host the middleware services.Docker was installed on the VM to simplify deployment and management of cloud services. Two primary containers were deployed which are: EMQX MQTT Broker and Node-RED

The EMQX broker was configured with authentication credentials and TLS support to secure MQTT communication. The broker endpoint accepts MQTT connections on port 8883.Node-RED was configured to subscribe to home/telemetry and receive telemetry messages from the broker. JSON processing nodes were used to parse incoming payloads before database insertion.The middleware layer acts as the central processing component responsible for message routing, device verification, and telemetry management.

2.3 Database Deployment

MongoDB Atlas was selected as the cloud database platform due to its scalability, availability, and support for time-series workloads. Two collections were implemented which is Device Metadata Collection used for stores authorized device information and serves as the authorization source for incoming devices. Besides that, Sensor Time-Series Collection: The sensor_timeseries collection stores telemetry data generated by the ESP32.The time-series collection is optimized for continuous sensor data ingestion and historical analysis.

2.4 Application Layer Deployment

MongoDB Charts was connected directly to the Atlas cluster to provide real-time visualization.The dashboard contains four visualizations which are Real-Time Intrusion Distance Monitor (Gauge Chart), Distance Over Time (Line Chart), IR Detection Frequency Chart and IoT Sensor Event Log (Data Table). These visualizations allow users to monitor intrusion activity and analyze historical sensor behaviour.

3. Cloud VM Migration Challenges

3.1 Migration from Local Deployment to Azure

The system was initially developed and tested on a local machine where Node-RED, MQTT connectivity, and MongoDB integration were functioning correctly. However, migrating the solution to Azure required rebuilding much of the middleware configuration.

Unlike the local environment, the cloud deployment required reconfiguring MQTT broker connections, database credentials, Node-RED flows, firewall settings, and external network access. Although the Node-RED flow could be exported from the local machine, all runtime configurations needed to be recreated and validated on the cloud platform. This increased deployment complexity and required extensive end-to-end testing before the system became operational.

3.2 Node-RED Authentication Configuration in Docker Container

One of the most significant challenges involved securing the Node-RED editor with username and password authentication.The laboratory guide assumes a standard Node-RED installation where the configuration file is stored in: /home/<user>/.node-red/settings.js. However, the deployed Node-RED instance was running inside a Docker container. As a result, modifications made to the configuration file on the Azure host machine had no effect on the active Node-RED service.After investigation, it was discovered that the actual configuration file used by the running Node-RED instance was located inside the container at: /data/settings.js. To modify the configuration, it was necessary to enter the container using: sudo docker exec -it nodered bash. The administrator authentication section was then manually added to the settings file using a hashed password generated through the Node-RED password utility. Finally, the container was restarted to apply the changes**.** This challenge highlighted the importance of understanding containerized application architectures and the distinction between host-level files and container-level files.

3.3 MQTT TLS Certificate Configuration

Another major challenge involved establishing a secure MQTT connection between Node-RED and the EMQX Cloud broker. The EMQX Cloud deployment required MQTT over TLS on port 8883. Initial connection attempts repeatedly failed despite the broker endpoint, username, and password being configured correctly..Further troubleshooting revealed that the MQTT client within Node-RED required the Certificate Authority (CA) certificate provided by EMQX Cloud. Without importing the CA certificate into the MQTT node configuration, TLS handshakes could not be successfully completed..After configuring the CA certificate within Node-RED, secure communication with the broker was successfully established. This issue demonstrated the importance of certificate management when deploying secure cloud-based MQTT systems.

3.4 Hostname and DNS Resolution Issues

Additional difficulties were encountered when configuring the MQTT broker hostname. Although the broker address appeared correct, several connection failures occurred due to hostname configuration errors. Because EMQX Cloud relies on fully qualified domain names rather than static IP addresses, even minor configuration mistakes prevented successful broker connections. Multiple rounds of testing were required to verify such as Broker hostname correctness, Port configuration, TLS settings and Authentication credentials. Once these parameters were verified, Node-RED successfully connected to the EMQX Cloud broker and telemetry messages were transmitted without interruption.

4. Security Verification

4.1 MQTT Authentication Verification

EMQX authentication was configured using username and password credentials. Valid credentials where Username: test and Password: 123. Verification was performed by attempting broker connections using both valid and invalid credentials. Result showed that authorized

clients successfully connected and exchanged messages. This confirmed that authentication controls were functioning correctly.

4.2 TLS Encryption Verification

All MQTT communication between the ESP32 and EMQX utilized TLS encryption through port 8883.Verification was performed by confirming successful TLS handshakes and secure message delivery between the client and broker. The encrypted channel prevents unauthorized interception of telemetry data during transmission.

4.2 Device Authorization Verification

Node-RED implements a verification workflow before inserting telemetry into MongoDB Atlas. The workflow performs the following steps. Firstly, receive MQTT message. Secondly, extract deviceId. Then, query the device_metadata collection. After that, verify device registration and active status. Lastly, allow insertion only if verification succeeds.Testing was performed using the registered device ESP32_001. Only devices present in the metadata collection were accepted for storage.This mechanism prevents unauthorized devices from polluting the database.

4.3 MongoDB Security Verification

MongoDB Atlas authentication was used to secure database access. Verification included successful authentication using valid credentials, rejection of invalid login attempts. And confirmation that telemetry data were written only after successful authentication.

5. Results and Discussion

The completed system successfully achieved all project objectives. The ESP32 reliably monitored environmental conditions and transmitted telemetry through MQTT. The cloud middleware layer successfully processed and validated incoming data before storage in MongoDB Atlas. The MongoDB Charts dashboard provided near real-time visualization of sensor activity, enabling continuous monitoring of intrusion events and sensor behaviour.The deployment demonstrates how modern cloud technologies can be combined to build scalable, secure, and efficient IoT applications.

6. Conclusion

This project successfully developed and deployed a complete cloud-based IoT home security monitoring platform. The solution integrates an ESP32 edge device, secure MQTT communication through EMQX, Node-RED middleware processing, MongoDB Atlas data storage, and MongoDB Charts visualization.The implementation demonstrated successful cloud deployment, device verification, secure communication, and real-time data analytics. The project provides a practical example of an end-to-end cyber-physical system and establishes a foundation for future enhancements such as mobile alerts, image-based intrusion detection, and advanced access control mechanisms.